

BACHELOR OF COMPUTER SCIENCE
FACULTY/SCHOOL OF SCHOOL OF COMPUTER SCIENCE
BINA NUSANTARA UNIVERSITY MEDAN
ASSESSMENT FORM

Course: COSC6070060 - Data Structure

Method of Assessment: Case Study

Semester/Academic Year: Even / 2025-2026

Name of Lecturer: CHARLES JHONY MANTHO SIANTURI, S.Kom, M.Kom

Date: 02 Feb - 13 Jun 2026

Class: All Class

Topic: Indonesia - English dictionary, at least 300 words data.

LAPORAN DOKUMENTASI PROJECT DATA STRUCTURE

Dibuat Oleh: Group 4

- **JOSE KELVIN STEVEN ANDREAN RAMBE - 2902742413**
- **GIOVANNI GOH - 2902689471**
- **RIVALDO HUTAGALUNG - 2902701804**

Tanggal Submission: TBD

Student Outcomes:

SO 2 - Mampu merancang, mengimplementasikan, dan mengevaluasi solusi berbasis komputasi untuk memenuhi serangkaian persyaratan komputasi dalam konteks ilmu computer

Able to design, implement, and evaluate a computing-based solution to meet a given set of computing requirements in the context of computer science

Learning Objectives:

LObj 2.2 - Mampu mengimplementasikan solusi berbasis komputasi untuk memenuhi serangkaian persyaratan komputasi tertentu dalam konteks ilmu computer

Able to implement a computing-based solution to meet a given set of computing requirements in the context of computer science

Learning Outcomes:

LO 3 - Apply data structures using C

1. Gambaran Umum Project

1.1 Apa itu IndoEng Dictionary?

IndoEng Dictionary adalah aplikasi kamus dwibahasa Indonesia-Inggris berbasis teks (console) yang dibangun menggunakan bahasa pemrograman C. Aplikasi ini memiliki fitur lengkap untuk mengelola 350+ pasangan kata dengan berbagai operasi pencarian dan manipulasi data. Berbeda dengan aplikasi kamus konvensional, project ini menerapkan pendekatan multi-struktur data untuk mengoptimalkan performa pada setiap fitur spesifiknya.

1.2 Spesifikasi Teknis Aplikasi

Komponen Teknis	Detail Spesifikasi
Bahasa Pemrograman	C (C99 Standard)
Platform UI/UX	Console / Terminal Interaktif
Compiler	GCC (GNU Compiler Collection)
Kapasitas Dataset	~350+ Pasang Kata Indonesia-Inggris
Arsitektur Sistem	Integrasi 6 Jenis Struktur Data Berbeda

2. Struktur File dan Organisasi Kode Program

Project ini diorganisasikan ke dalam beberapa direktori untuk memisahkan antara spesifikasi, konfigurasi build, implementasi logika utama, dan file executable:

```
indoeng-dictionary/
├── SPEC.md           # Spesifikasi mendalam project
├── Makefile          # Konfigurasi otomatisasi build sistem
├── src/
│   ├── main.c        # Entry point utama & manajemen menu interaktif
│   ├── dictionary.c   # Logika koordinasi utama (Dictionary Manager)
│   ├── dictionary.h   # Definisi header dan API Dictionary Manager
│   ├── data_structures.c # Implementasi konkret dari 6 struktur data
│   ├── data_structures.h # Deklarasi tipe data, enum, dan konstanta
│   └── word_data.c    # Dataset statis berisi 350+ kata awal
└── bin/
    └── indoeng-dictionary.exe # File biner hasil kompilasi (executable)
```

2.1 Modul Kontrol Utama (`main.c`)

Modul ini bertindak sebagai pengatur alur program interaktif. Fitur-fitur utama yang disediakan dalam menu console meliputi:

1. **View All Words:** Navigasi untuk melihat seluruh kata di dalam kamus dengan sistem pagination dan filter huruf alfabet.
2. **Search Indonesian:** Pencarian kata dalam bahasa Indonesia dengan memanfaatkan struktur data BST.
3. **Search English:** Pencarian kata dalam bahasa Inggris menggunakan Hash Table.
4. **Prefix Search:** Fitur pengetikan prediktif atau autocomplete berbasis Trie.
5. **Add New Word:** Penambahan kosakata baru ke dalam sistem dengan validasi duplikasi otomatis.
6. **Update Word:** Pembaruan entri kata secara dinamis per komponen field field yang dipilih user.
7. **Delete Word:** Penghapusan kata secara permanen dari sistem.
8. **Word of the Day:** Fitur penampil kata acak harian yang memanfaatkan antrean sirkular.
9. **Search History:** Menampilkan daftar pencarian terakhir pengguna.
10. **Undo Last Action:** Fasilitas pembatalan aksi terakhir (Add/Update/Delete) berbasis LIFO Stack.
11. **Statistics:** Menu penampil metrik dan visualisasi statistik distribusi kata dalam kamus.

2.2 Arsitektur Komponen Dictionary Manager (`dictionary.c`)

Komponen ini mengkoordinasikan seluruh struktur data agar tetap sinkron. Berikut adalah deklarasi objek DictionaryManager:

C/C++

```
typedef struct {  
    BSTNode* bst_root;           // Root BST untuk lookup alfabetis  
    HashTable* hash_english;     // Hash Table untuk lookup konstan  
    TrieNode* trie_root;        // Root Trie untuk autocomplete  
    LinkedList* search_history;  // Doubly Linked List untuk history  
    Stack* undo_stack;           // Stack untuk manajemen Undo  
    Queue* word_queue;           // Queue untuk Word of the Day  
    int total_words;             // Counter total kata  
    int word_of_day_index;       // Indeks pelacak harian  
    WordEntry last_searched;     // Buffer pencarian terakhir  
};
```

```

    int has_last_searched;           // Flag status pencarian
} DictionaryManager;

```

3. Implementasi Mendalam 6 Struktur Data

3.1 Binary Search Tree (BST)

Tujuan Utama: Penyimpanan kata secara alfabetis & pencarian $O(\log n)$

Struktur Node:

```

C/C++
typedef struct BSTNode {
    WordEntry word;
    struct BSTNode* left;
    struct BSTNode* right;
} BSTNode;

```

Karakteristik:

- **Left subtree:** Kata yang lebih kecil (A-Z)
- **Right subtree:** Kata yang lebih besar
- **Pencarian:** Seperti mencari di kamus (buka halaman tengah, bandingkan)
- **Complexitas:** $O(\log n)$ rata-rata, $O(n)$ worst case

Operasi utama:

Fungsi	Deskripsi
bst_create_node()	Buat node baru
bst_insert()	Sisipkan kata baru
bst_search()	Cari kata berdasarkan Indonesia
bst_delete()	Hapus kata
bst_find_min()	Cari node dengan nilai terkecil (successor)
bst_update_key()	Update key (indonesian) di BST
bst_inorder()	Traversing in-order (A-Z)
bst_destroy()	Bebaskan memori

bst_count()	Hitung jumlah node
-------------	--------------------

Implementasi pencarian:

C/C++

```
BSTNode* bst_search(BSTNode* root, const char* indonesian) {
    if (root == NULL) return NULL;
    int cmp = strcasecmp(indonesian, root->word.indonesian);
    if (cmp == 0) return root;
    else if (cmp < 0) return bst_search(root->left, indonesian);
    else return bst_search(root->right, indonesian);
}
```

Implementasi penyisipan:

C/C++

```
BSTNode* bst_insert(BSTNode* root, WordEntry word) {
    if (root == NULL) return bst_create_node(word);
    int cmp = strcasecmp(word.indonesian, root->word.indonesian);
    if (cmp < 0) root->left = bst_insert(root->left, word);
    else if (cmp > 0) root->right = bst_insert(root->right, word);
    else root->word = word; // Update jika sudah ada
    return root;
}
```

Implementasi penghapusan:

```
BSTNode* bst_delete(BSTNode* root, const char* indonesian) {
    // Kasus 1: Node tidak ditemukan &rarr; return NULL
    // Kasus 2: Node adalah leaf (tanpa anak) &rarr; free, return NULL
    // Kasus 3: Node punya satu anak &rarr; replace dengan anak, free
    // Kasus 4: Node punya dua anak &rarr; cari successor (min dari right subtree)
}
```

3.2 Hash Table

Tujuan Utama: Pencarian $O(1)$ berdasarkan kata Inggris

Struktur:

```
#define HASH_TABLE_SIZE 1000
```

```
typedef struct HashNode {  
    WordEntry word;  
    struct HashNode* next; // Chaining untuk collision  
} HashNode;
```

```
typedef struct {  
    HashNode* buckets[HASH_TABLE_SIZE];  
    int size;  
} HashTable;
```

Karakteristik:

- Menggunakan **chaining** untuk menangani collision
- **Hash function:** Menggunakan metode polynomial rolling hash
- **Key:** Kata Inggris (case-insensitive)
- **Complexitas:** $O(1)$ average case

Operasi utama:

Fungsi	Deskripsi
hash_create()	Buat hash table baru
hash_insert()	Sisipkan kata
hash_search()	Cari kata Inggris
hash_delete()	Hapus kata (return WordEntry*)
hash_remove()	Hapus kata (tanpa return)
hash_destroy()	Bebaskan memori
hash_size()	Mendapatkan ukuran

Implementasi pencarian:

```
WordEntry* hash_search(HashTable* ht, const char* english) {  
    unsigned int index = hash_function(english);  
    HashNode* current = ht->buckets[index];  
    while (current != NULL) {  
        if (strcasecmp(current->word.english, english) == 0) {  
            return &(current->word); // Ketemu!  
        }  
        current = current->next;  
    }  
    return NULL;  
}
```

```

    }
    current = current->next;
}
return NULL; // Tidak ditemukan
}

```

3.3 Trie (Prefix Tree)

Tujuan Utama: Autocomplete & pencarian prefix (awalan kata)

Struktur:

```
#define ALPHABET_SIZE 26
```

```

typedef struct TrieNode {
    struct TrieNode* children[ALPHABET_SIZE];
    int is_end_of_word;
    WordEntry word; // Simpan word entry di node akhir
    int word_count; // Counter untuk prefix
} TrieNode;

```

Karakteristik:

- Setiap node memiliki 26 children (a-z)
- **Path:** Merepresentasikan prefix kata
- **Node akhir:** Menandai akhir sebuah kata
- **Complexitas:** $O(m)$ where m = panjang prefix

Visualisasi Trie:

Contoh kata: "makan", "maju", "kecil"

Root

```

├── m
|   ├── a
|   |   ├── k
|   |   |   ├── a
|   |   |   └── n (END: makan)
|   |   └── j
|   └── u (END: maju)
└── i

```

```

|   └─ n
|   └─ u
|   └─ m
|   └─ (END: minum)
└─ k
   └─ e
      └─ c
         └─ i
            └─ l (END: kecil)

```

Operasi utama:

Fungsi	Deskripsi
trie_create_node()	Buat node baru
trie_insert()	Sisipkan kata
trie_search_prefix()	Cari prefix node
trie_get_all_with_prefix()	Ambil semua kata dengan prefix
trie_collect_all()	Helper - rekursif collect semua kata
trie_destroy()	Bebaskan memori

Implementasi penyisipan:

```

void trie_insert(TrieNode* root, const char* word, WordEntry entry) {
    TrieNode* current = root;
    while (*word) {
        int index = tolower(*word) - 'a';
        if (current->children[index] == NULL) {
            current->children[index] = trie_create_node();
        }
        current = current->children[index];
        current->word_count++;
        word++;
    }
    current->is_end_of_word = 1;
    current->word = entry; // Simpan word entry
}

```


Implementasi pencarian prefix:

```
TrieNode* trie_search_prefix(TrieNode* root, const char* prefix) {
    TrieNode* current = root;
    while (*prefix) {
        int index = tolower(*prefix) - 'a';
        if (current->children[index] == NULL) {
            return NULL; // Prefix tidak ditemukan
        }
        current = current->children[index];
        prefix++;
    }
    return current; // Node prefix ditemukan
}
```

Implementasi get all with prefix:

```
void trie_get_all_with_prefix(TrieNode* root, const char* prefix,
WordEntry** results, int* count) {
    TrieNode* prefix_node = trie_search_prefix(root, prefix);
    if (prefix_node == NULL) {
        *results = NULL;
        *count = 0;
        return;
    }
    int capacity = 10;
    *results = (WordEntry*)malloc(capacity * sizeof(WordEntry));
    *count = 0;
    trie_collect_all(prefix_node, results, count, &capacity);
}
```

3.4 Doubly Linked List (Riwayat Pencarian)

Tujuan Utama: Menyimpan riwayat pencarian (history)

Struktur:

```
typedef struct ListNode {
    WordEntry word;
```

```

struct ListNode* next;
struct ListNode* prev;
} ListNode;

typedef struct {
    ListNode* head;
    ListNode* tail;
    int size;
    int max_size; // Batas maksimal (20 entries)
} LinkedList;

```

Karakteristik:

- **Doubly Linked List** (ada prev & next)
- **FIFO (Most Recently Used):** Item terbaru di depan, terlama di belakang
- **Auto-trimming:** Jika size > max_size, hapus tail
- **Duplicate handling:** Jika kata sudah ada di list, pindahkan ke head (update posisi)

Operasi utama:

Fungsi	Deskripsi
list_create()	Buat list baru (dengan max_size)
list_insert_front()	Sisipkan di depan
list_insert_back()	Sisipkan di belakang (FIFO)
list_search()	Cari dalam list
list_remove()	Hapus dari list
list_display()	Tampilkan semua
list_destroy()	Bebaskan memori

Implementasi sisip depan (dengan duplicate handling):

```

void list_insert_front(LinkedList* list, WordEntry word) {
    // 1. Cek duplicate - jika ada, pindahkan ke head
    // 2. Jika list penuh, hapus tail terlebih dahulu
    // 3. Buat node baru, set sebagai head

    ListNode* new_node = (ListNode*)malloc(sizeof(ListNode));
    new_node->word = word;
    new_node->next = list->head;

```

```

new_node->prev = NULL;

if (list->head != NULL) {
    list->head->prev = new_node;
}

list->head = new_node;

if (list->tail == NULL) {
    list->tail = new_node;
}

list->size++;
}

```

3.5 Stack (Mekanisme Undo)

Tujuan Utama: Operasi undo (membatalkan aksi)

Struktur:

```
#define MAX_STACK_SIZE 100
```

```

typedef struct {
    WordEntry entries[MAX_STACK_SIZE];
    int top; // -1 jika kosong
    int max_size;
} Stack;

```

Karakteristik:

- **LIFO (Last In First Out):** Item terakhir masuk, pertama keluar
- Fixed size array: Maksimal 100 undo actions
- **Operasi:** Push (tambah), Pop (ambil), Peek (lihat)

Operasi utama:

Fungsi	Deskripsi
stack_create()	Buat stack baru
stack_push()	Tambah item ke top
stack_pop()	Ambil item dari top
stack_peek()	Lihat item di top
stack_is_empty()	Cek apakah kosong

stack_is_full()	Cek apakah penuh
-----------------	------------------

Penggunaan dalam Undo (Enhanced dengan Marker):

- Menggunakan prefix khusus (**@ADD:**, **@UPDATE:**, **@DELETE:**) pada field *indonesian* di *WordEntry* yang dimasukkan ke stack.
- Saat **add kata** → push dengan marker **@ADD:nama_kata**, aksi undo berupa penghapusan kata.
- Saat **update kata** → push dengan marker **@UPDATE:nama_kata + data kata lama**, aksi undo berupa pemulihan record data lama.
- Saat **delete kata** → push with marker **@DELETE:nama_kata + data kata**, aksi undo berupa memasukkan kembali kata tersebut ke sistem.

3.6 Circular Queue (Word of the Day)

Tujuan Utama: Fitur "Word of the Day" & shuffle kata

Struktur:

```
#define MAX_QUEUE_SIZE 1000
```

```
typedef struct {
    WordEntry entries[MAX_QUEUE_SIZE];
    int front; // Index depan
    int rear; // Index belakang (-1 jika kosong)
    int size; // Jumlah item saat ini
    int max_size;
} Queue;
```

Karakteristik:

- **Circular Queue:** Menggunakan modulo untuk wrap-around
- **FIFO (First In First Out):** Item pertama masuk, pertama keluar
- **Shuffle:** Menggunakan Fisher-Yates algorithm untuk random

Operasi utama:

Fungsi	Deskripsi
queue_create()	Buat queue baru
queue_enqueue()	Tambah item ke belakang
queue_dequeue()	Ambil item dari depan
queue_peek()	Lihat item di depan
queue_get_random()	Ambil kata acak

queue_shuffle()	Acak urutan queue
queue_is_empty()	Cek apakah kosong
queue_is_full()	Cek apakah penuh

4. Alur Kerja Aplikasi

4.1 Inisialisasi

main()

↓

dict_create() → Buat DictionaryManager

- └─ bst_root = NULL
- └─ hash_english = hash_create()
- └─ trie_root = trie_create_node()
- └─ search_history = list_create(20)
- └─ undo_stack = stack_create(50)
- └─ word_queue = queue_create(1000)

↓

dict_load_initial_data() → Muat 350+ kata

- └─ Buat WordEntry dari dataset
- └─ dict_add_word() untuk setiap kata
- └─ Reset undo_stack (kosongkan)

4.2 Menambah Kata

dict_add_word(dict, word)

- |
- └─ bst_search() → Cek apakah sudah ada
- | └─ Jika ada → return 0 (gagal)
- |
- └─ bst_insert() → Sisipkan ke BST
- └─ hash_insert() → Sisipkan ke Hash Table
- └─ trie_insert() → Sisipkan ke Trie
- └─ queue_enqueue() → Tambah ke queue

```
└── total_words++
|
└── push undo marker (@ADD:nama_kata)
```

4.3 Update Kata

dict_update_word(dict, indonesian, new_word)

```
|
└── bst_search() → Cari kata lama
|
└── Simpan kata lama untuk undo
|
└── bst_delete() → Hapus kata lama
└── bst_insert() → Sisipkan kata baru
└── hash_delete() + hash_insert() → Update Hash
└── Rebuild Trie (destroy + rebuild)
|
└── push undo marker (@UPDATE:nama_kata + data lama)
|
└── return 1 (berhasil)
```

4.4 Hapus Kata

dict_delete_word(dict, indonesian)

```
|
└── bst_search() → Cari kata
|
└── Simpan kata untuk undo
|
└── bst_delete() → Hapus dari BST
└── hash_delete() → Hapus dari Hash
└── Rebuild Trie
└── Total words--
|
└── push undo marker (@DELETE:nama_kata + data kata)
|
```

- └─ return 1 (berhasil)

4.5 Pencarian Indonesia (BST)

dict_search_indonesian(dict, "makan")

- |
 - └─ bst_search(bst_root, "makan")
 - └─ Recursive: kiri jika lebih kecil, kanan jika lebih besar
- |
 - └─ Jika ketemu:
 - └─ list_insert_front() → Tambah ke history
 - └─ dict_set_last_searched() → Simpan kata terakhir
- |
 - └─ return WordEntry* atau NULL

4.6 Pencarian Inggris (Hash Table)

dict_search_english(dict, "eat")

- |
 - └─ hash_function("eat") → hitung index
- |
 - └─ hash_search() → traverse linked list di bucket
 - └─ Bandingkan case-insensitive
- |
 - └─ Jika ketemu:
 - └─ list_insert_front() → Tambah ke history
 - └─ dict_set_last_searched() → Simpan kata terakhir
- |
 - └─ return WordEntry* atau NULL

4.7 Autocomplete (Trie)

dict_search_prefix(dict, "ma")

- |
 - └─ trie_search_prefix(trie_root, "ma")
 - └─ Traverse: m → a
 - └─ Return node "ma"

```

|
|— trie_get_all_with_prefix()
|   — Panggil trie_collect_all() secara rekursif
|   — DFS dari node "ma" → collect semua is_end_of_word
|
|— return array WordEntry[] + count

```

4.8 Undo (Enhanced dengan Marker)

dict_undo(dict)

```

|
|— stack_pop() → Ambil undo marker dari stack
|
|— Parse marker prefix (@ADD: / @UPDATE: / @DELETE:)
|
|— Jika @ADD: → dict_delete_word_no_undo() → Hapus kata (undo add)
|
|— Jika @UPDATE: → dict_add_word() → Restore kata lama
|
|— Jika @DELETE: → dict_add_word() → Restore kata (undo delete)
|
|— Tampilkan pesan sukses dengan nama kata
|
|— Tunggu user tekan Enter untuk kembali ke menu

```

5. Analisis Kompleksitas Algoritma

Berikut merupakan matriks perbandingan performa teoretis dari keenam jenis struktur data yang diintegrasikan di dalam aplikasi:

Struktur Data	Insert	Search	Delete	Use Case
BST	$O(\log n)$	$O(\log n)$	$O(\log n)$	Lookup Indo (Terurut)
Hash Table	$O(1)$	$O(1)$	$O(1)$	Lookup Inggris Cepat

Trie	$O(m)$	$O(m)$	$O(m)$	Autocomplete
Linked List	$O(1)$	$O(n)$	$O(n)$	Log History
Stack	$O(1)$	-	$O(1)$	Mekanisme Undo
Queue	$O(1)$	$O(1)$	$O(1)$	Word of the Day

**Catatan: Nilai kompleksitas pada Hash Table dihitung berdasarkan skenario kasus rata-rata (Average Case). Nilai m merepresentasikan panjang string karakter kata.*

6. Sinkronisasi Data dan Integrasi Sistem

Salah satu aspek paling krusial dalam project ini adalah **Sinkronisasi Data Mutlak**. Mengingat sistem menggunakan 6 struktur data yang berbeda secara bersamaan, setiap kali pengguna melakukan operasi manipulasi (CRUD), fungsi koordinator wajib mengeksekusi pembaruan ke seluruh modul struktur data yang bersangkutan.

Sebagai contoh, ketika operasi `dict_add_word()` dipanggil, sistem secara sekuensial akan:

1. Memeriksa status duplikasi string pada modul **BST**.
2. Melakukan alokasi dan penyisipan node pada pohon **BST**.
3. Memasukkan referensi entri data ke dalam bucket array **Hash Table**.
4. Memetakan setiap runtunan karakter huruf ke dalam struktur node **Trie**.
5. Memasukkan record entri ke dalam antrean sirkular **Queue**.
6. Membuat salinan penanda riwayat transaksi data (marker) ke dalam **Stack** undo.

Pendekatan sinkronisasi terpusat ini menjamin keaslian data (data integrity) dan mencegah terjadinya ketidakcocokan informasi di dalam aplikasi.

7. Kategori Kata dalam Dataset

7.1 Everyday (Sehari-hari)

Kata-kata umum yang digunakan dalam percakapan sehari-hari.

- Contoh: makan, minum, buku, rumah

7.2 Formal (Resmi)

Kata-kata yang digunakan dalam konteks formal atau akademis.

- Contoh: implementasi, konfigurasi, evaluasi

7.3 Slang (Gaul)

Kata-kata kekinian yang populer di kalangan anak muda.

- Contoh: baper, gabut, ngopi, cuan

7.4 Technical (Teknis)

Kata-kata dalam domain teknologi dan komputer.

- Contoh: algoritma, variabel, debugging
-

8. Fitur Unggulan

8.1 Multi-Structure Integration

Demonstrasikan bagaimana 6 struktur data bekerja BERSAMA:

User input "makan"

BST: Apakah ada di dictionary?

Hash: Simpan terjemahan "eat"

Trie: Index untuk autocomplete "ma..."

History: Catat pencarian

Stack: Siapkan untuk undo

Queue: Word of the Day

8.2 Data Synchronization

Saat menambah/menghapus/mengupdate kata, SEMUA struktur data harus tersinkron:

```
int dict_add_word(DictionaryManager* dict, WordEntry word) {  
    // 1. Cek apakah sudah ada di BST  
    if (bst_search(dict->bst_root, word.indonesian) != NULL) {  
        return 0; // Gagal - sudah ada  
    }
```

```
    // 2. Insert ke BST
```

```
    dict->bst_root = bst_insert(dict->bst_root, word);
```

```
    // 3. Insert ke Hash Table
```

```
    hash_insert(dict->hash_english, word);
```

```
    // 4. Insert ke Trie
```

```
    trie_insert(dict->trie_root, word.indonesian, word);
```

// 5. Enqueue ke Queue

```
queue_enqueue(dict->word_queue, word);
```

// 6. Update counter

```
dict->total_words++;
```

// 7. Push undo marker

```
WordEntry undo_marker = word;
```

```
snprintf(marker_prefix, "@ADD:%s", word.indonesian);
```

```
strncpy(undo_marker.indonesian, marker_prefix, MAX_WORD_LEN - 1);
```

```
stack_push(dict->undo_stack, undo_marker);
```

```
return 1;
```

```
}
```

8.3 Undo Mechanism (Enhanced dengan Marker)

Stack-based undo dengan marker untuk identifikasi aksi:

- **@ADD:** → Undo add = delete kata
- **@UPDATE:** → Undo update = restore kata lama
- **@DELETE:** → Undo delete = restore kata yang dihapus

Pesan sukses menunjukkan aksi apa yang di-undo.

8.4 Fitur Update Fleksibel

Pada fitur Update Word:

1. Pilih kata yang akan diupdate
2. Pilih field mana yang ingin diubah (1-7)
3. Tekan Enter untuk tidak mengubah, input simbol - untuk mengosongkan
4. Setelah selesai, tekan **0** untuk simpan dan keluar

8.5 Last Searched Word Display

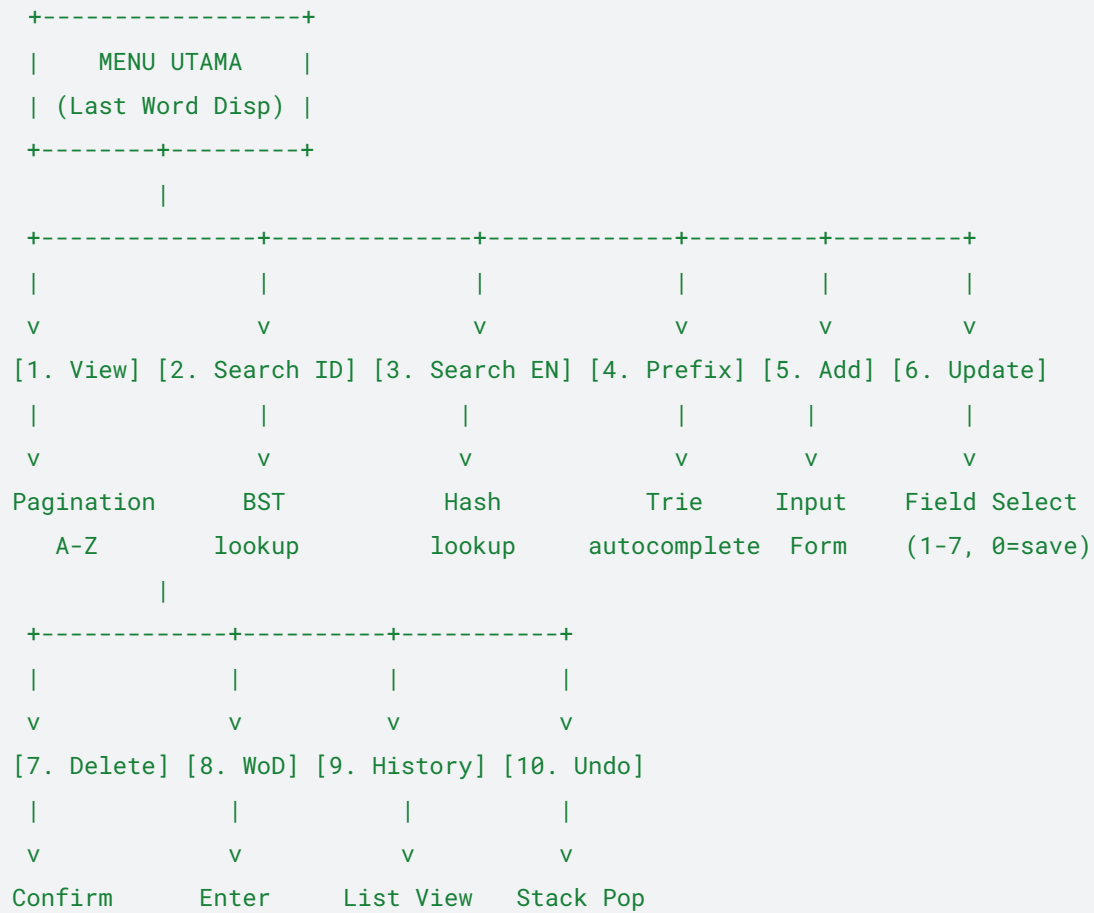
Kata terakhir yang dicari ditampilkan di atas menu utama setiap kali menu utama ditampilkan.

8.6 View All Words dengan Filter

- Pagination: 20 kata per halaman
 - Filter A-Z: Tekan huruf A-Z untuk filter berdasarkan awalan
 - Pilih kata: Tekan nomor (1-N) untuk lihat detail
 - Navigasi halaman: /1, /2, dst
-

9. Diagram Alur Menu

None



10. Contoh Output Program

Menu Utama:

None

```
=====
=                                KATA TERAKHIR                                =
=====
| Indonesian : rumah                |
| English    : house                |
| POS        : Noun                 |
+=====+
```

None

```
+-----+
| INDONESIAN - ENGLISH DICTIONARY |
| (IndoEng)                        |
+-----+
| 1. View All Words - Paginated A-Z |
| 2. Search Word (Indonesian) - BST Lookup |
| 3. Search Word (English) - Hash Table Lookup |
| 4. Prefix Search (Autocomplete) - Trie Lookup |
| 5. Add New Word - Create |
| 6. Update Word - Update |
| 7. Delete Word - Delete |
| 8. Word of the Day - Random Pick |
| 9. Search History - Recent Searches |
| 10. Undo Last Action - Stack-based |
| 11. Statistics - Dictionary Stats |
| 0. Exit |
```

+-----+

Masukkan pilihan:

Contoh Undo Berhasil:

None

[Sukses] Undo berhasil! Menambahkan kata 'rumah' telah dikembalikan.

[Tekan Enter untuk kembali ke menu utama]

Pilihan:

Contoh Update Word:

None

+-----+

| UPDATE: rumah |

+-----+

| 1. Indonesian : rumah |

| 2. English : house |

| 3. POS : Noun |

| 4. Category : Everyday |

| 5. Pronounce : [haus] |

| 6. Definition : bangunan tempat tinggal |

| 7. Example : Saya pergi ke rumah. |

+-----+

| |

| [1-7] Pilih field yang akan diubah |

| [0] Simpan dan keluar |

| |

+-----+

Pilihan:

Contoh View All Words:

None

+-----+

| FILTER: [*] [A] [B] [C] ... |

+-----+

Filter: * | Halaman 1/8 | Total: 150 kata

+-----+

No	Indonesian	English	POS	Category
----	------------	---------	-----	----------

+-----+

1	algoritma	algorithm	Noun	Technical
---	-----------	-----------	------	-----------

2	arsitektur	architecture	Noun	Technical
---	------------	--------------	------	-----------

...	...	...	...	...
-----	-----	-----	-----	-----

+-----+

+-----+

| [1-150] Pilih Kata | [A-Z] Awalan | [/1-8] Halaman |

| [0] Tampilkan Semua | [<] Menu Utama |

+-----+

Pilihan:

11. Tips untuk Presentasi

Sesi Demonya:

1. **Tunjukkan menu utama** - 11 opsi fitur + last searched word
2. **Demo View All Words** - Pagination, filter A-Z, pilih kata (tekan 1-N)
3. **Demo pencarian Indonesia** - Gunakan BST, tunjukkan proses

4. **Demo pencarian Inggris** - Gunakan Hash, tunjukkan $O(1)$
5. **Demo autocomplete** - Ketik "ma", tunjukkan semua kata "ma..."
6. **Demo Add Word** - Validasi kata sudah ada, tekan Enter
7. **Demo Update Word** - Pilih field satu per satu, tekan Enter, simbol -
8. **Demo Delete Word** - Konfirmasi, sukses tekan Enter
9. **Demo Word of the Day** - Kata acak, tekan Enter
10. **Demo undo** - Add/Update/Delete, lalu undo, kata kembali/dihapus
11. **Demo statistics** - Tunjukkan distribusi POS & Category

Poin Penting untuk Disampaikan:

- **Kenapa 6 struktur data?** Masing-masing punya tujuan spesifik
- **Trade-offs:** BST (ordered) vs Hash (fast lookup)
- **Enhanced Undo:** Marker-based approach untuk identifikasi aksi
- **UX:** Tekan Enter setelah setiap operasi untuk konfirmasi
- **Update Fleksibel:** Pilih field satu per satu, tekan Enter untuk skip
- **Real-world application:** Dictionary sederhana tapi lengkap
- **Memory management:** Allokasi & dealokasi memori